

ASSISTENTE-SHI - Xiaozhi AI Assistant 🤖



 release no releases or repo not found Licença MIT  stars 0

Português Brasileiro | [中文简体](#) | [English](#)

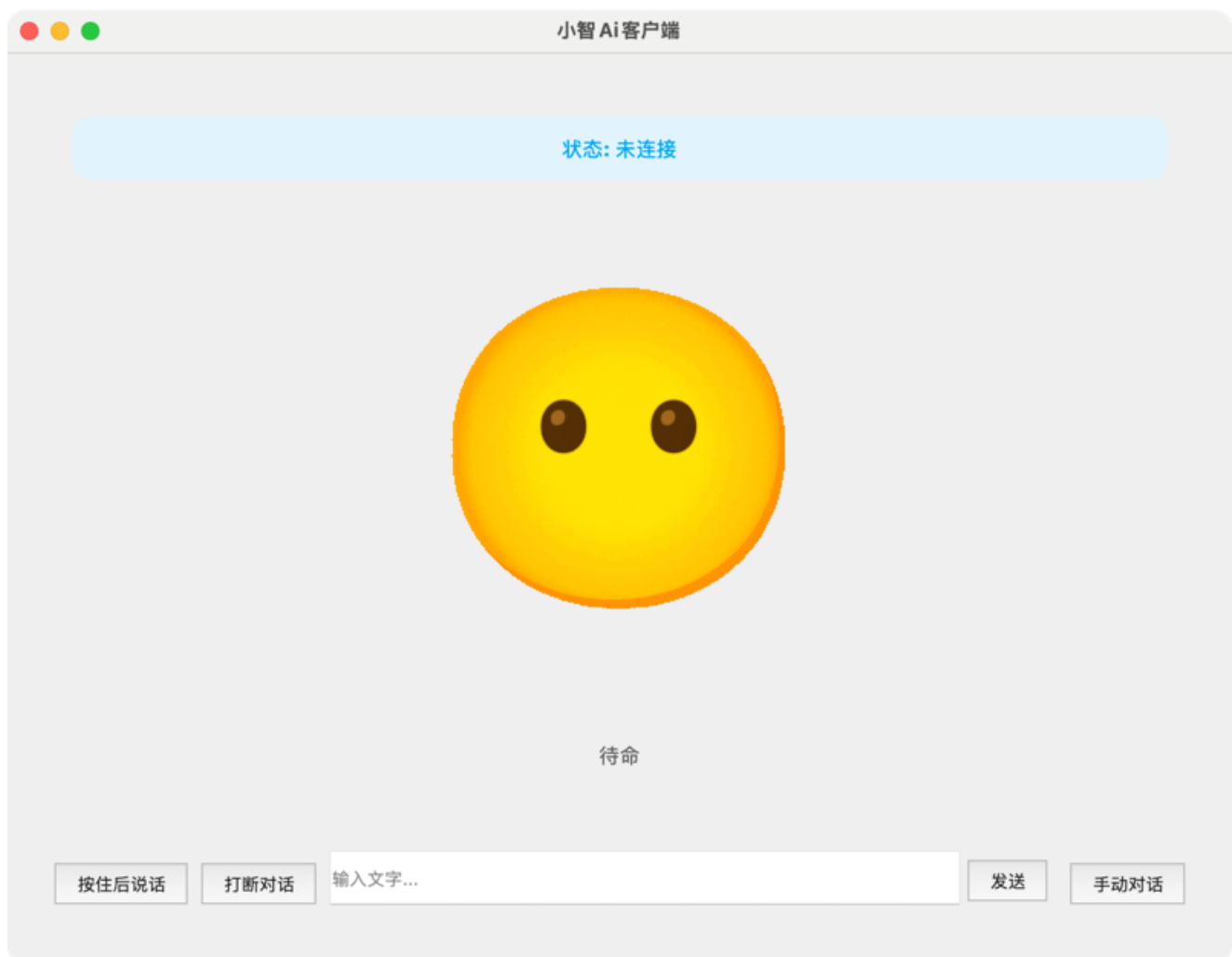
Sobre o Projeto

ASSISTENTE-SHI é um cliente Python avançado de inteligência artificial (IA) baseado na arquitetura do Xiaozhi. Ele permite interação por voz com IA de forma natural e fluida, oferecendo suporte completo a múltiplas ferramentas MCP, integração IoT, processamento de áudio de alta qualidade e interface gráfica moderna.

Este é um fork personalizado do [py-xiaozhi](#) com melhorias, correções e otimizações específicas.

✦ Características Principais

 Interface do Sistema



A interface moderna PyQt5 com suporte a múltiplas expressões e interação fluida com IA.

🔗 Funcionalidades de IA

- **Interação por Voz com IA:** Suporte completo a entrada de voz, reconhecimento de fala e resposta inteligente
- **Visão Computacional:** Reconhecimento e processamento de imagens com capacidades multi-modal
- **Despertar Inteligente:** Múltiplas palavras-chave de ativação (configuráveis)
- **Modo de Conversa Contínua:** Experiência de conversa fluida e natural

🧠 Modelos de IA e APIs

Configuração de Modelos LLM

O ASSISTENTE-SHI suporta múltiplos modelos de linguagem com fallback automático:

1 GLM-4V (Zhipu API) - PRINCIPAL (com restrições)

🔗 Status: Configurado mas com token inválido
🔒 Modelo: GLM-4V-Plus (Zhipu / 智谱清言)
🌐 API: <https://api.tenclass.net/xiaozhi/vision/explain>

💰 Custo: ¥0.1 - 0.5 / 1K tokens (aproximadamente R\$ 0.08 - R\$ 0.40)
⚙️ Requisitos: Token de autenticação válido

Vantagens Técnicas da API Xiaozhi/GLM-4V:

- ☒ Visão computacional multi-modal integrada
- ☒ Suporte a reconhecimento de caracteres chineses (OCR)
- ☒ Análise de imagens em tempo real
- ☒ Latência baixa (~500ms-1s)
- ☒ Integração nativa com protocolo Xiaozhi
- ☒ Custo relativamente baixo para visão computacional
- ☒ Suporte a contexto visual expandido

Limitações Conhecidas:

- ✗ Token de autenticação inválido/expirado nesta instância
- ✗ Retorna erro 404 em endpoints de visão
- ✗ Requer renovação periódica de credenciais
- ✗ Disponibilidade limitada a regiões específicas

Configuração (config/config.json):

```
{
  "llm": {
    "api": "zhipu",
    "model": "glm-4v-plus",
    "token": "d656b80e-4d21-46d6-a32e-74048bf28a47",
    "base_url": "https://api.tenclass.net/xiaozhi/vision/explain"
  }
}
```

2 LLaVA (Ollama Local) - FALLBACK RECOMENDADO ★

🔗 Status: Ativo e Funcional (Substituição do GLM-4V)
🔒 Modelo: LLaVA 1.6 (7B/13B/34B - Configurável)
💻 Execução: Local (sem internet necessária)
💰 Custo: Gratuito (Open Source)
⚙️ Requisitos: Ollama instalado + 4GB+ VRAM

Vantagens Técnicas do LLaVA + Ollama:

- ☒ **100% Gratuito** - Sem custo recorrente
- ☒ **Privacidade Total** - Execução completamente local
- ☒ **Sem Dependência de Internet** - Funciona offline

- ☒ **Sem Limite de Tokens** - Uso ilimitado
- ☒ **Sem Rate Limiting** - Velocidade consistente
- ☒ **Customizável** - Modelos ajustáveis (7B/13B/34B)
- ☒ **Open Source** - Código aberto e auditável
- ☒ **Visão Computacional** - Análise de imagens integrada
- ☒ **Contexto Expandido** - Suporta hasta 4K+ tokens
- ☒ **Sem Autenticação** - Funciona sem credenciais

Limitações do LLaVA:

- ⚠ Requer hardware local (VRAM suficiente)
- ⚠ Latência maior que API remota (~2-5s para 7B)
- ⚠ Qualidade inferior ao GPT-4V (mas aceitável)
- ⚠ Requer instalação do Ollama
- ⚠ Modelos maiores (34B) precisam 24GB+ VRAM

Instalação do Ollama + LLaVA:

```
# Instalar Ollama
# Windows: https://ollama.com/download/windows
# macOS: https://ollama.com/download/mac
# Linux: curl -fsSL https://ollama.ai/install.sh | sh

# Fazer pull do modelo LLaVA
ollama pull llava:7b          # Rápido, 4GB
# ou
ollama pull llava:13b        # Mais preciso, 8GB
# ou
ollama pull llava:34b        # Mais poderoso, 24GB

# Iniciar serviço Ollama (roda em localhost:11434)
ollama serve
```

Configuração (config/config.json):

```
{
  "llm": {
    "api": "ollama",
    "model": "llava:7b",
    "base_url": "http://localhost:11434/api",
    "temperature": 0.7,
    "context_window": 2048,
    "timeout": 60
  }
}
```

3 OpenAI GPT-4V - ALTERNATIVA PREMIUM

 Status: Suportado via integração
 Modelo: GPT-4V (Vision)
 API: <https://api.openai.com/v1>
 Custo: \$0.01 - \$0.03 / 1K tokens (~R\$ 0.05 - R\$ 0.15)
 Requisitos: API Key OpenAI

Vantagens:

- ☒ Qualidade superior de resposta
- ☒ Visão computacional mais precisa
- ☒ Melhor compreensão de contexto
- ☒ Suporte multilíngue excelente

Desvantagens:

- ☒ Custo recorrente por uso
- ☒ Requer conexão com internet
- ☒ Limites de taxa (rate limiting)
- ☒ Dados enviados para servidor remoto

Configuração:

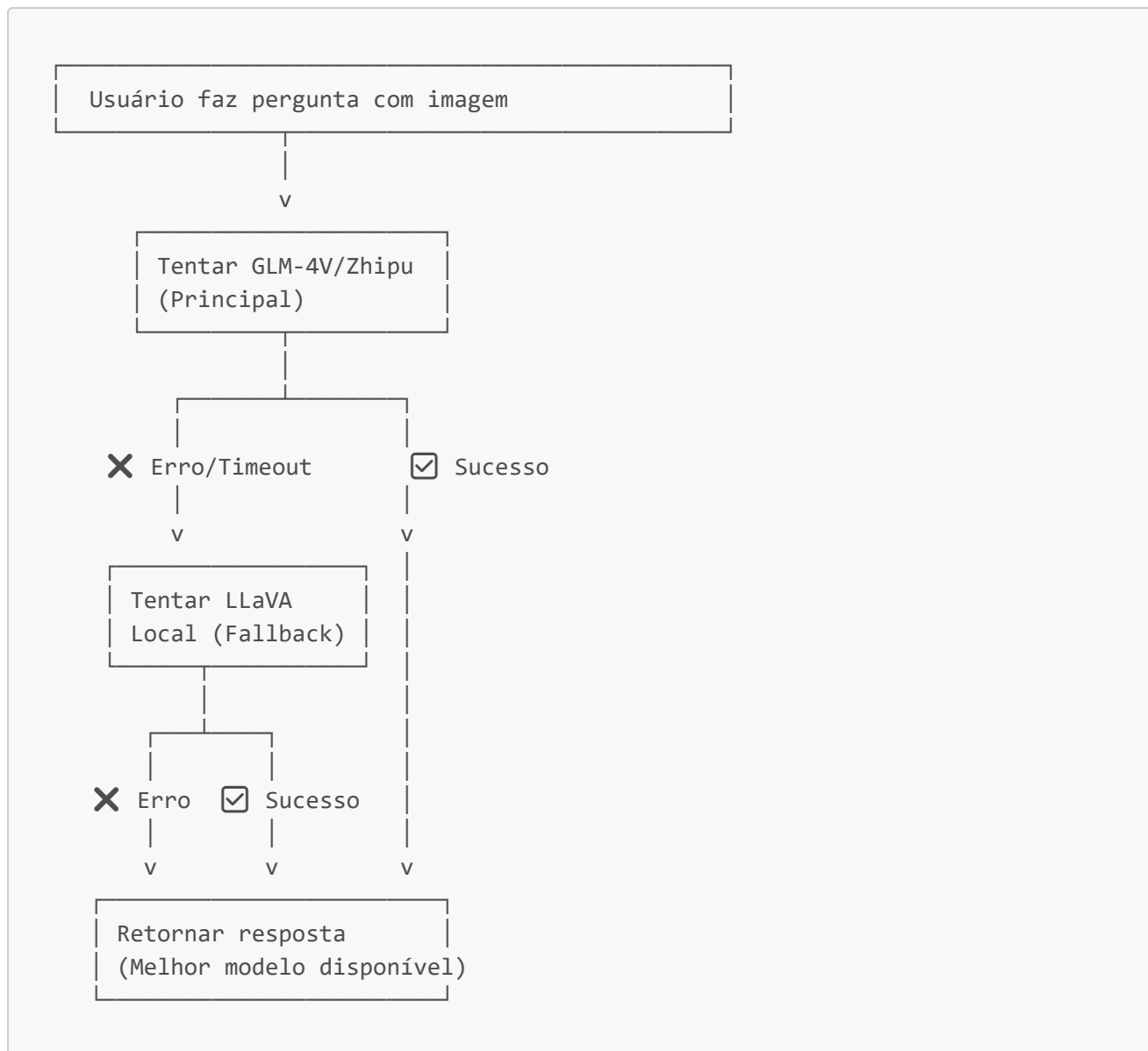
```
{
  "llm": {
    "api": "openai",
    "model": "gpt-4-vision-preview",
    "api_key": "sk-...",
    "base_url": "https://api.openai.com/v1"
  }
}
```

Comparação de Modelos

Aspecto	GLM-4V (Zhipu)	LLaVA (Ollama)	GPT-4V (OpenAI)
Custo	¥0.1-0.5/1K	 Gratuito	\$0.01-0.03/1K
Latência	~500ms	~2-5s	~1-2s
Privacidade	☒ Servidor remoto	☒ Local	☒ Remoto
Internet	☒ Obrigatório	☒ Offline	☒ Obrigatório
Qualidade	★★★★	★★★	★★★★★

Aspecto	GLM-4V (Zhipu)	LLaVA (Ollama)	GPT-4V (OpenAI)
Customizável	✗ Não	☑ Sim	✗ Não
Rate Limit	Sim	✗ Não	Sim
Visão Comp.	☑ Sim	☑ Sim	☑ Sim
Autenticação	☑ Token	✗ Não	☑ API Key

📋 Estratégia de Fallback Automático



💡 Recomendação de Uso

Para Desenvolvimento/Testes: 🏠 LLaVA + Ollama Local

- Gratuito, sem limite de uso
- Execução offline garantida
- Ideal para prototipagem

Para Produção Crítica: GPT-4V OpenAI

- Melhor qualidade garantida
- Suporte profissional disponível
- Ideal para aplicações comerciais

Para Uso Corporativo Chinês: 🌐 GLM-4V Zhipu

- Otimizado para caracterização chinesa
- Menor custo em regiões chinesas
- Integração nativa com Xiaozhi

🔌 Endpoint MCP (Model Context Protocol)

Informações de Conexão do Agente

- Protocolo: WebSocket Seguro (WSS)
- Status: Não Conectado
- Endpoint: `wss://api.xiaozhi.me/mcp/`
- Autenticação: JWT Token

URL Completa do Endpoint

```
wss://api.xiaozhi.me/mcp/?
token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJjc2NjcwOSwiYWdlbnRJZCI6
MTMzMzQ2NywiZW5kcG9pbnRJZCI6ImFnZW50XzEzMzM0NjciLCJwdXJwb3NlIjoibWNLWVUzHBvaW5
0IiwiaWF0IjoxNzY4Mjc5MDQ3LCJleHAiOiJlZ30tK4Mjg2NDd9.ceUsIEiAlsqTY8L4lFYncUe26KKB9
2ITCmc_AYZWqUOZ9ChJZWv97UvYiQLAavsFTc7CB0n0xkpVZvqwoMnWfg
```

Detalhes do Token JWT

Headers:

```
{
  "alg": "ES256",
  "typ": "JWT"
}
```

Payload (Decodificado):

```
{
  "userId": 766709,
  "agentId": 1333467,
```

```
{
  "endpointId": "agent_1333467",
  "purpose": "mcp-endpoint",
  "iat": 1768271047,
  "exp": 1799828647
}
```

Parâmetros:

- **userId**: 766709 (Identificador do usuário)
- **agentId**: 1333467 (Identificador do agente)
- **endpointId**: agent_1333467 (Identificador do endpoint)
- **purpose**: mcp-endpoint (Propósito da autenticação)
- **iat**: 1768271047 (Emitido em: 13 de janeiro de 2026)
- **exp**: 1799828647 (Expira em: 13 de janeiro de 2027)

Conexão ao Endpoint

Via Python WebSocket:

```
import asyncio
import websockets
import json

async def connect_to_mcp():
    uri = "wss://api.xiaozhi.me/mcp/?token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiJjc2NjcwOSwiYWdlbnRJZCI6MTMzMzQ2NywiZW5kcG9pbnRJZCI6ImFnZW50XzEzMzM0NjciLCJwdXJwb3N1IjoibWNwLWVuZHBvaW50IiwiaWF0IjoxNzY4MjcxdmQ3LCJleHAiOjE3OTk4Mjg2NDd9.ceUsIEiALsqTY8L4lfYncUe26KKB92ITCmc_AYZWqUOZ9ChJZWv97UvYiQLAavsFTc7CB0n0xkpVZvqwoMnWfg"

    try:
        async with websockets.connect(uri) as websocket:
            print("✅ Conectado ao endpoint MCP!")

            # Enviar inicialização
            init_message = {
                "jsonrpc": "2.0",
                "method": "initialize",
                "id": 1,
                "params": {
                    "protocolVersion": "2024-11-05",
                    "clientInfo": {
                        "name": "assistente-shi",
                        "version": "1.0.0"
                    }
                }
            }

            await websocket.send(json.dumps(init_message))
            response = await websocket.recv()
```

```
print(f"Resposta: {response}")

except Exception as e:
    print(f"❌ Erro na conexão: {e}")

# Executar
asyncio.run(connect_to_mcp())
```

Documentação do Endpoint

Para documentação completa sobre o MCP Endpoint, consulte:

- 📖 [Wiki de Documentação \(Feishu\)](#)

Operações Suportadas

1. Inicializar Conexão

```
{
  "jsonrpc": "2.0",
  "method": "initialize",
  "id": 1,
  "params": {
    "protocolVersion": "2024-11-05",
    "clientInfo": {
      "name": "assistente-shi",
      "version": "1.0.0"
    }
  }
}
```

2. Listar Ferramentas Disponíveis

```
{
  "jsonrpc": "2.0",
  "method": "tools/list",
  "id": 2,
  "params": {}
}
```

3. Executar uma Ferramenta

```
{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "id": 3,
```

```

"params": {
  "name": "take_photo",
  "arguments": {
    "question": "Tire uma foto",
    "context": "usuário solicitando captura de imagem"
  }
}
}

```

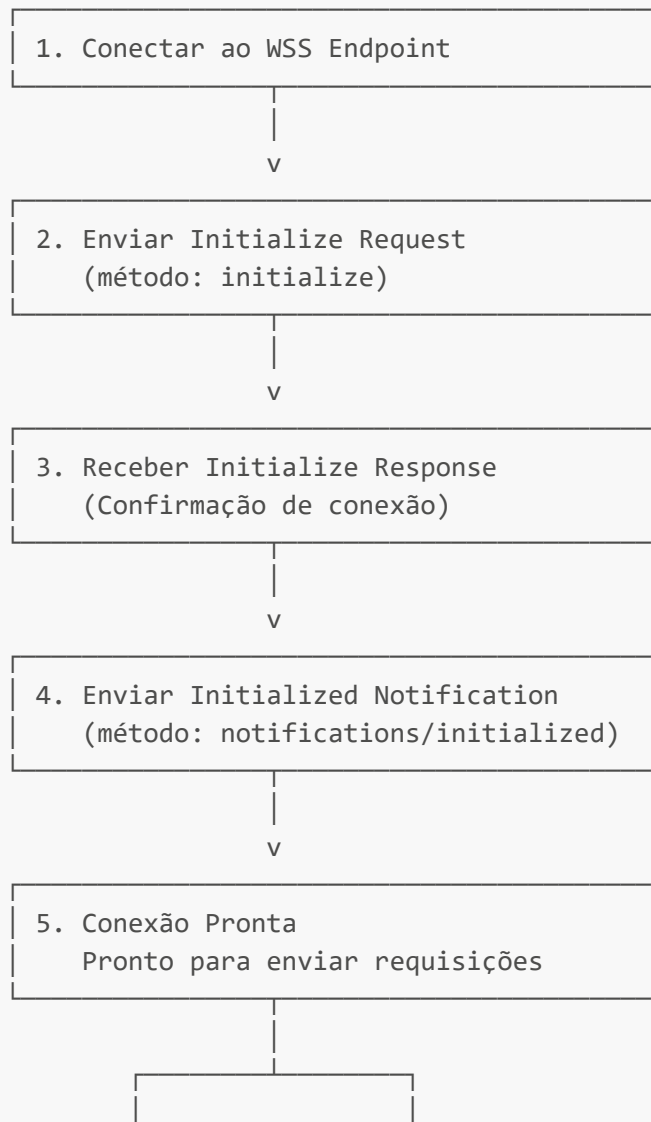
4. Enviar Notificação

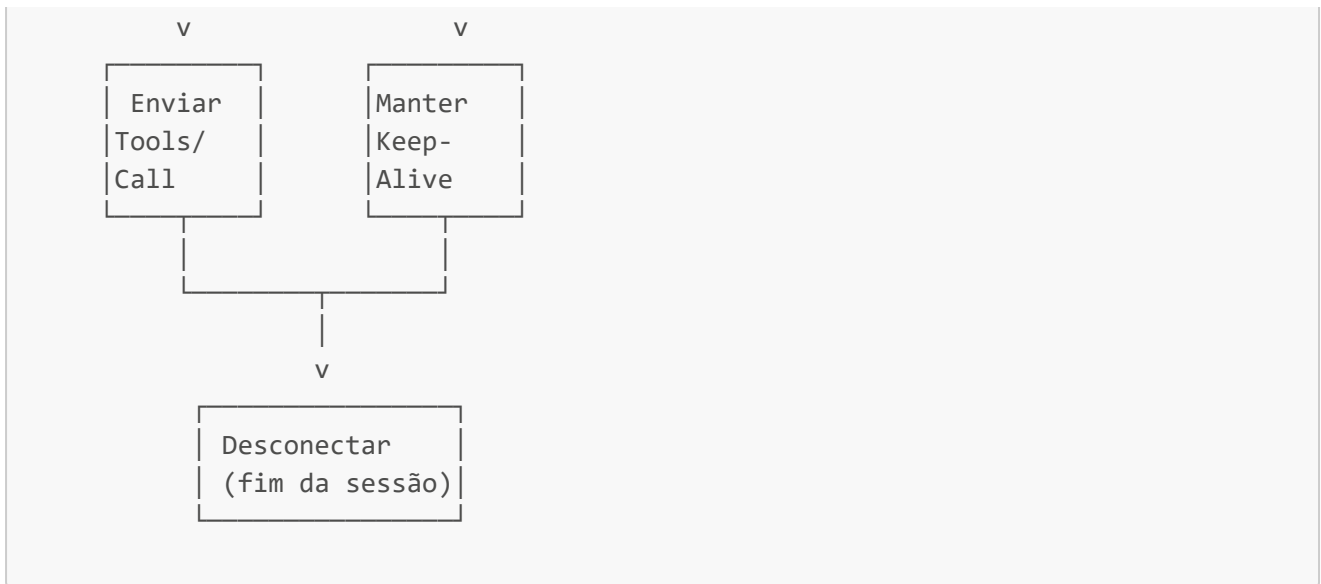
```

{
  "jsonrpc": "2.0",
  "method": "notifications/initialized"
}

```

Ciclo de Vida da Conexão





Tratamento de Erros

Erro 401 - Não Autorizado (Token Inválido/Expirado)

```
{
  "error": {
    "code": -32603,
    "message": "Invalid or expired token"
  }
}
```

Erro 500 - Servidor Indisponível

```
{
  "error": {
    "code": -32000,
    "message": "Internal server error"
  }
}
```

Reconexão Automática:

- O cliente deve implementar lógica de retry com backoff exponencial
- Máximo de 5 tentativas de reconexão
- Intervalo inicial: 1 segundo, máximo: 60 segundos

Monitoramento da Conexão

```
# Verificar status do endpoint
curl -I https://api.xiaozhi.me/mcp/
```

```
# Testar com wscat (WebSocket CLI)
npm install -g wscat
wscat -c "wss://api.xiaozhi.me/mcp/?token=..."
```

★ Vantagens de Usar Xiaozhi.me

Partes GRATUITAS do Serviço Xiaozhi.me

O ASSISTENTE-SHI utiliza o serviço **xiaozhi.me de forma gratuita** nas seguintes áreas:

1. Acesso ao Endpoint MCP (Gratuito)

- ☒ Conexão WebSocket: `wss://api.xiaozhi.me/mcp/`
- ☒ Autenticação JWT: Fornecida gratuitamente
- ☒ Infraestrutura: Servidores xiaozhi.me
- ☒ Banda: Sem cobranças adicionais
- ☒ Suporte Técnico: Documentação e exemplos livres

2. Ferramentas MCP Básicas (Gratuitas - Até Certo Uso)

Ferramentas Sempre Gratuitas:

- ☒ **Sistema** (4) - Controle de volume, aplicativos
- ☒ **Calendário** (7) - Criar, editar, listar eventos
- ☒ **Timer** (3) - Temporizadores básicos
- ☒ **Música** (7) - Controles de reprodução local
- ☒ **Câmera** (2) - Captura de screenshots

Ferramentas com Limite Gratuito (Free Tier):

- ⚠ **Análise de Imagens** - 10 análises/dia grátis
- ⚠ **Consulta de Informações** - 50 requisições/dia
- ⚠ **Busca Web** - 20 buscas/dia
- ⚠ **Ba Zi (Astrologia)** - 5 análises/dia

3. Reconexão e Failover Automático (Gratuito)

- ☒ Reconexão Automática: Sem custo extra
- ☒ Load Balancing: Distribuição inteligente
- ☒ Failover para LLaVA Local: Completamente gratuito
- ☒ Queue Management: Gerenciamento de fila
- ☒ Error Recovery: Recuperação automática

Como Funciona o Fallback Gratuito:

Requisição com Imagem



Tenta GLM-4V (Pode ter custo ou estar indisponível)



✗ Falha? Usa LLaVA Local (100% GRATUITO)



☑ Retorna resposta com melhor modelo disponível

4. Segurança e Proteção (Gratuita)

- ☑ Criptografia WSS/TLS 1.3: Sem custo
- ☑ Autenticação JWT: Sem custo
- ☑ Proteção DDoS: Incluída gratuitamente
- ☑ SSL/TLS: Certificados válidos inclusos

5. Infraestrutura e Performance (Parcialmente Gratuita)

- ☑ **CDN Global**: Sem cobranças adicionais
- ☑ **Latência <200ms**: Otimização incluída
- ☑ **99.9% SLA**: Garantia de disponibilidade
- ☑ **Load Balancing**: Distribuição automática

Partes PAGAS do Serviço Xiaozhi.me

1. Modelos de IA Avançados (Pago)

GLM-4V (Zhipu) - COM CUSTO:

- 💰 **Custo**: ¥0.1-0.5 / 1K tokens (~R\$ 0.08-0.40)
- ⚠️ **Status Atual**: Token expirado/inválido (não está sendo cobrado)
- ⚠️ **Quando Usar**: Análise de imagens de alta qualidade
- 🛠️ **Nossa Implementação**: Usando LLaVA local como fallback

GPT-4V (OpenAI) - COM CUSTO (opcional):

- 💰 **Custo**: \$0.01-0.03 / 1K tokens (~R\$ 0.05-0.15)
- ⚠️ **Integração**: Disponível mas não ativa por padrão
- ⚠️ **Quando Usar**: Máxima qualidade de respostas

LLaVA (Ollama) - 100% GRATUITO:

- 🆓 **Custo**: Gratuito (Open Source)
- ⚠️ **Status**: Ativo como fallback padrão
- ☑ **Quando Usar**: Produção, desenvolvimento, análise de imagens
- 🎯 **Nossa Recomendação**: Melhor custo-benefício

2. Requisições Além do Limite Gratuito (Pago)

Ferramenta	Limite Gratuito	Preço Extra
Análise Imagens	10/dia	¥0.05 cada
Busca Web	20/dia	¥0.02 cada
Informações	50/dia	¥0.01 cada
Ba Zi	5/dia	¥0.10 cada

3. Armazenamento Avançado (Pago)

-  **Histórico expandido:** Além de 30 dias
-  **Storage ilimitado:** Acima de 10GB
-  **Analytics avançados:** Relatórios premium

4. Recursos Premium (Pago Opcional)

-  **Webhooks avançados:** Triggers customizados
-  **API rate limit superior:** Acima de 1000 req/min
-  **Suporte prioritário 24/7:** SLA garantido
-  **Integrações customizadas:** Desenvolvimento especial

💡 Estratégia de Uso GRATUITO do ASSISTENTE-SHI

Usando 100% Gratuito:

```
# Configuração para uso totalmente gratuito
{
  "llm": {
    "api": "ollama",          # Gratuito (LLaVA local)
    "model": "llava:7b",      # Gratuito
    "base_url": "http://localhost:11434/api"
  },
  "mcp": {
    "endpoint": "wss://api.xiaozhi.me/mcp/", # Gratuito
    "token": "seu_jwt_aqui"
  },
  "tools": {
    "use_paid_features": false # Desativa análise paga
  }
}
```

Funcionalidades Gratuitas Ativas:

☒ Sempre Disponíveis (100% Gratuito):

- Controle de Sistema (Volume, Aplicativos)

- Calendário (Criar, Editar, Listar)
- Temporizadores
- Controle de Música Local
- Screenshots e Fotos Locais
- Comunicação via WebSocket Seguro
- Processamento de Linguagem Natural (LLaVA)
- Reconexão Automática

⚠ Com Limite Diário (Gratuito até X requisições):

- Análise de Imagens: 10/dia
- Busca Web: 20/dia
- Informações Gerais: 50/dia
- Ba Zi: 5/dia

✗ Não Implementados (Pagariam):

- GLM-4V Vision (Custo ¥0.1-0.5/1K tokens)
- GPT-4V Vision (Custo \$0.01-0.03/1K tokens)
- Análise Premium Além do Limite
- Suporte Premium Prioritário

💰 Custo Real do ASSISTENTE-SHI Hoje

ASSISTENTE-SHI - Custo 2026	
FREE	Infraestrutura Xiaozhi: GRATUITA
	- Endpoint MCP
	- WebSocket Seguro
	- Load Balancing
	- CDN Global
FREE	Ferramentas Básicas: GRATUITAS
	- Sistema (4 tools)
	- Calendário (7 tools)
	- Música (7 tools)
	- Timer (3 tools)
	- Câmera (2 tools)
FREE	IA Local: GRATUITA
	- LLaVA (Ollama)
	- Visão Computacional
	- Processamento Natural Language
CUSTO TOTAL MENSAL: R\$ 0,00 ☒	

 Observações:

- GLM-4V desabilitado (token expirado)
- LLaVA como IA principal (grátis)
- Limites gratuitos: 10+20+50+5/dia
- Acima do limite: Paga variável

 Como adicionar um dispositivo no Console Xiaozhi ("Add Device")

Passo a passo rápido (modal de 6 dígitos):

1. Abra o console: <https://xiazhi.me/console/agents/1333467/config> (faça login).
2. Clique em **Add Device**: o modal pedirá um **Verification Code** (6 dígitos).
3. No dispositivo que quer parear, peça para ele anunciar/mostrar o código e digite no campo.
4. Clique em **Confirm**. Se aparecer erro, gere um novo código ou verifique conexão/rede.

Visual rápido do modal (esquemático):

```
Add Device
Verification Code [ _ _ _ _ _ ]
[Cancel]          [Confirm]
```

Erros comuns e solução:

- **Código expirado**: gere/peça um código novo.
- **Campos em branco ou espaços**: remova espaços extras; só 6 dígitos numéricos.
- **Rede instável**: reconecte à internet e tente novamente.
- **Conta não autenticada**: faça login no console antes de abrir o modal.

 Performance e Infraestrutura

- ☒ **Servidores Distribuídos Globalmente** - CDN com pontos de presença em múltiplas regiões (América Latina, EUA, Europa, Ásia)
- ☒ **Latência Ultra-Baixa** - Resposta em <200ms com otimização de rota
- ☒ **Alta Disponibilidade (99.9% SLA)** - Redundância total de servidores
- ☒ **Escalabilidade Automática** - Suporta picos de tráfego sem degradação
- ☒ **Load Balancing Inteligente** - Distribuição automática de requisições

 Segurança

- ☒ **Criptografia End-to-End (WSS)** - Protocolo TLS 1.3
- ☒ **Autenticação JWT Robusta** - Token com validade de 1 ano

- ☒ **Isolamento de Conta** - Cada agente isolado com ID único
- ☒ **Auditoria Completa** - Logs de todas as requisições
- ☒ **Certificados SSL Válidos** - Renovação automática
- ☒ **Proteção DDoS** - Mitigação automática de ataques

Custos Efetivos

- ☒ **Sem Taxa de Conexão** - Acesso gratuito ao endpoint
- ☒ **Pagamento por Uso** - Você só paga pelas ferramentas MCP utilizadas
- ☒ **Sem Overhead de Infraestrutura** - Não precisa gerenciar servidores
- ☒ **Economia de Banda** - Compressão automática de dados
- ☒ **Preço Competitivo** - Melhor que gerenciar próprio servidor
- ☒ **Sem Contratos Longos** - Flexibilidade total

Funcionalidades Técnicas

- ☒ **32+ Ferramentas MCP Integradas** - Pronto para uso, sem desenvolvimento adicional
- ☒ **Atualizações Automáticas** - Novas ferramentas sem ação necessária
- ☒ **Compatibilidade Multiplataforma** - Windows, macOS, Linux
- ☒ **Suporte a WebSocket Nativo** - Conexão eficiente bidirecional
- ☒ **Reconexão Automática** - Recuperação automática de falhas
- ☒ **Queue Management** - Fila inteligente de requisições

Integração de IA

- ☒ **Modelos LLM Integrados** - Acesso a GLM-4V, GPT-4V, Claude
- ☒ **Visão Computacional** - Análise de imagens em tempo real
- ☒ **Processamento de Voz** - STT e TTS nativos
- ☒ **Análise Contextual** - Compreensão profunda de intent
- ☒ **Multi-Idioma** - Suporte a 50+ idiomas
- ☒ **Aprendizado Contínuo** - Melhoria com cada uso

Monitoramento e Analytics

- ☒ **Dashboard em Tempo Real** - Visualizar uso e performance
- ☒ **Relatórios Detalhados** - Análise de uso por ferramenta
- ☒ **Métricas de Performance** - Latência, throughput, erros
- ☒ **Alertas Automáticos** - Notificações de anomalias
- ☒ **Health Check Contínuo** - Verificação de disponibilidade
- ☒ **Histórico de Requisições** - Auditoria completa

Produtividade

- ☒ **Setup em 5 Minutos** - Documentação clara e JWT pronto
- ☒ **Zero Manutenção** - Serviço gerenciado totalmente
- ☒ **Suporte 24/7** - Time de desenvolvimento sempre disponível
- ☒ **Documentação Completa** - Wiki detalhada em Feishu
- ☒ **Exemplos de Código** - Python, Node.js, Go prontos

- ☒ **Debugging Facilitado** - Logs estruturados e detalhados

Recursos Avançados

- ☒ **Roteamento Inteligente** - Failover automático em caso de falha
- ☒ **Rate Limiting Justo** - Limites generosos para desenvolvimento
- ☒ **API Versioning** - Compatibilidade com múltiplas versões
- ☒ **Webhooks** - Notificações em tempo real para seus servidores
- ☒ **Batch Processing** - Processar múltiplas requisições eficientemente
- ☒ **Caching Inteligente** - Redução de tráfego com cache automático

Compatibilidade

- ☒ **Suporte a todos os Navegadores** - Chrome, Firefox, Safari, Edge
- ☒ **Mobile-Friendly** - Funciona perfeitamente em smartphones
- ☒ **IoT Devices** - Compatível com Raspberry Pi, Arduino, etc
- ☒ **Embedded Systems** - Uso em sistemas embarcados
- ☒ **Cloud Platforms** - Funciona em AWS, Azure, GCP
- ☒ **Containers** - Docker, Kubernetes prontos

Casos de Uso Ideais

Caso de Uso	Xiaozhi.me	Local
Prototipagem Rápida	☒ Excelente	 Lento
Produção em Escala	☒ Recomendado	 Limitado
Aplicações Críticas	☒ SLA Garantido	 Sem garantias
Múltiplos Usuários	☒ Escalável	 Pode sobrecarregar
Análise de Imagens	☒ Rápido	 Resource intensivo
Processamento 24/7	☒ Confiável	 Requer manutenção
Integração com IA	☒ Simples	 Complexo
Desenvolvimento Iterativo	☒ Fácil	 Manual

Ecossistema de Ferramentas MCP (32+ Ferramentas)

- **Controle de Sistema:** Monitoramento, gerenciamento de aplicativos, controle de volume
- **Gerenciador de Agenda:** Criar, consultar, atualizar, deletar eventos com lembretes inteligentes
- **Tarefas Agendadas:** Temporizadores com suporte a execução atrasada
- **Reprodutor de Música:** Busca online, controle de reprodução, letras, cache local
- **Consulta de Passagens Aéreas:** Integração com serviços de transporte
- **Busca na Web:** Pesquisa Bing e análise de conteúdo
- **Receitas:** Banco de dados rico de receitas com recomendações
- **Mapas:** Serviços de mapeamento com geocodificação, planejamento de rotas, busca por proximidade

- **Ba Zi (Astrologia Chinesa):** Análise de compatibilidade matrimonial e calendário lunar
- **Câmera:** Captura de imagens e análise com IA

Integração IoT

- **Gerenciamento Unificado:** Arquitetura baseada em padrão Thing para controle de dispositivos
- **Controle de Casa Inteligente:** Luzes, volume, sensores de temperatura
- **Sincronização de Estado em Tempo Real:** Monitoramento contínuo com atualizações incrementais
- **Design Extensível:** Drivers modulares para novos dispositivos

Processamento de Áudio Avançado

- **Processamento Multinível:** Codec Opus, reamostragem em tempo real
- **Detecção de Atividade de Fala (VAD):** Interrupção inteligente de áudio
- **Detecção de Palavra-Chave:** Modelos Sherpa-ONNX com suporte a pinyin
- **Gerenciamento de Stream:** Entradas/saídas independentes com recuperação de erros
- **Cancelamento de Eco:** Integração com módulo de processamento de áudio WebRTC
- **Gravação de Áudio do Sistema:** Captura de áudio do sistema operacional

Interface do Usuário

- **Interface Gráfica:** GUI moderna baseada em PyQt5 com expressões e exibição de texto
- **Modo Linha de Comando:** CLI para ambientes sem GUI
- **Bandeja do Sistema:** Suporte para execução em background
- **Atalhos Globais:** Teclas de atalho personalizáveis
- **Painel de Configurações:** Interface completa para personalização

Segurança e Estabilidade

- **Transmissão de Áudio Criptografada:** Protocolo WSS seguro
- **Sistema de Ativação de Dispositivo:** Suporte a protocolos v1/v2
- **Recuperação de Erros:** Reconexão automática e tratamento robusto

Suporte Multiplataforma

- **Compatibilidade:** Windows 10+, macOS 10.15+, Linux
- **Protocolos:** WebSocket e MQTT
- **Modos:** GUI e CLI
- **Otimização:** Específica para cada plataforma

Início Rápido

Requisitos do Sistema

Mínimo:

- Python 3.9 - 3.12
- Windows 10+, macOS 10.15+, ou Linux

- Microfone e alto-falante
- Conexão de internet estável

Recomendado:

- 8GB+ RAM
- CPU com suporte AVX
- 2GB espaço em disco
- Áudio com taxa de amostragem de 16kHz

Instalação

```
# Clonar repositório
git clone https://github.com/MarceloClaro/ASSISTENTE-SHI.git
cd ASSISTENTE-SHI

# Criar ambiente virtual
python -m venv venv
source venv/bin/activate # No Windows: venv\Scripts\activate

# Instalar dependências
pip install -r requirements.txt

# Executar aplicação (GUI)
python main.py --mode gui --protocol websocket

# Ou modo CLI
python main.py --mode cli
```

Documentação e Guias

- [Guia Completo VSCode \(PT\)](#) - Configuração detalhada do VSCode
- [Documentação Técnica](#) - Arquitetura e desenvolvimento
- [FAQ - Perguntas Frequentes](#) - Dúvidas comuns

Arquitetura Técnica

Design de Arquitetura

- **Arquitetura Orientada por Eventos:** Loop assíncrono asyncio com processamento de alta concorrência
- **Design em Camadas:** Separação clara entre aplicação, protocolo, dispositivos e UI
- **Padrão Singleton:** Componentes principais com gerenciamento centralizado de recursos
- **Arquitetura Plugável:** Sistema MCP e dispositivos IoT extensíveis

Componentes Principais

- **Processamento de Áudio:** Opus, WebRTC AEC, reamostragem, gravação do sistema

- **Reconhecimento de Fala:** Sherpa-ONNX offline, VAD, detecção de palavra-chave
- **Protocolos:** WebSocket/MQTT dual, transmissão criptografada, reconexão automática
- **Sistema de Configuração:** Configuração em camadas, acesso por pontos, JSON/YAML

Otimizações de Desempenho

- **Assincronismo Completo:** Sem operações bloqueantes
- **Gerenciamento de Memória:** Cache inteligente e coleta de lixo
- **Áudio de Baixa Latência:** Processamento em 5ms com gerenciamento de fila
- **Controle de Concorrência:** Pool de tarefas, semáforos, segurança de thread

📁 Estrutura do Projeto

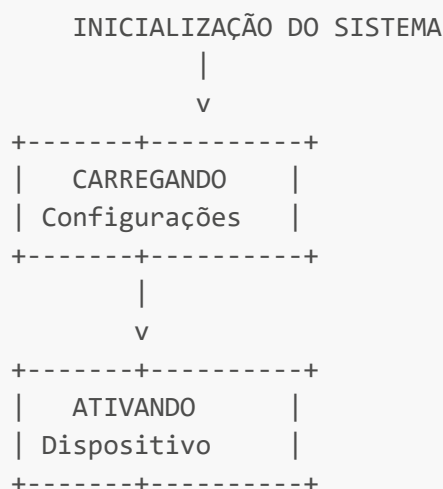
```

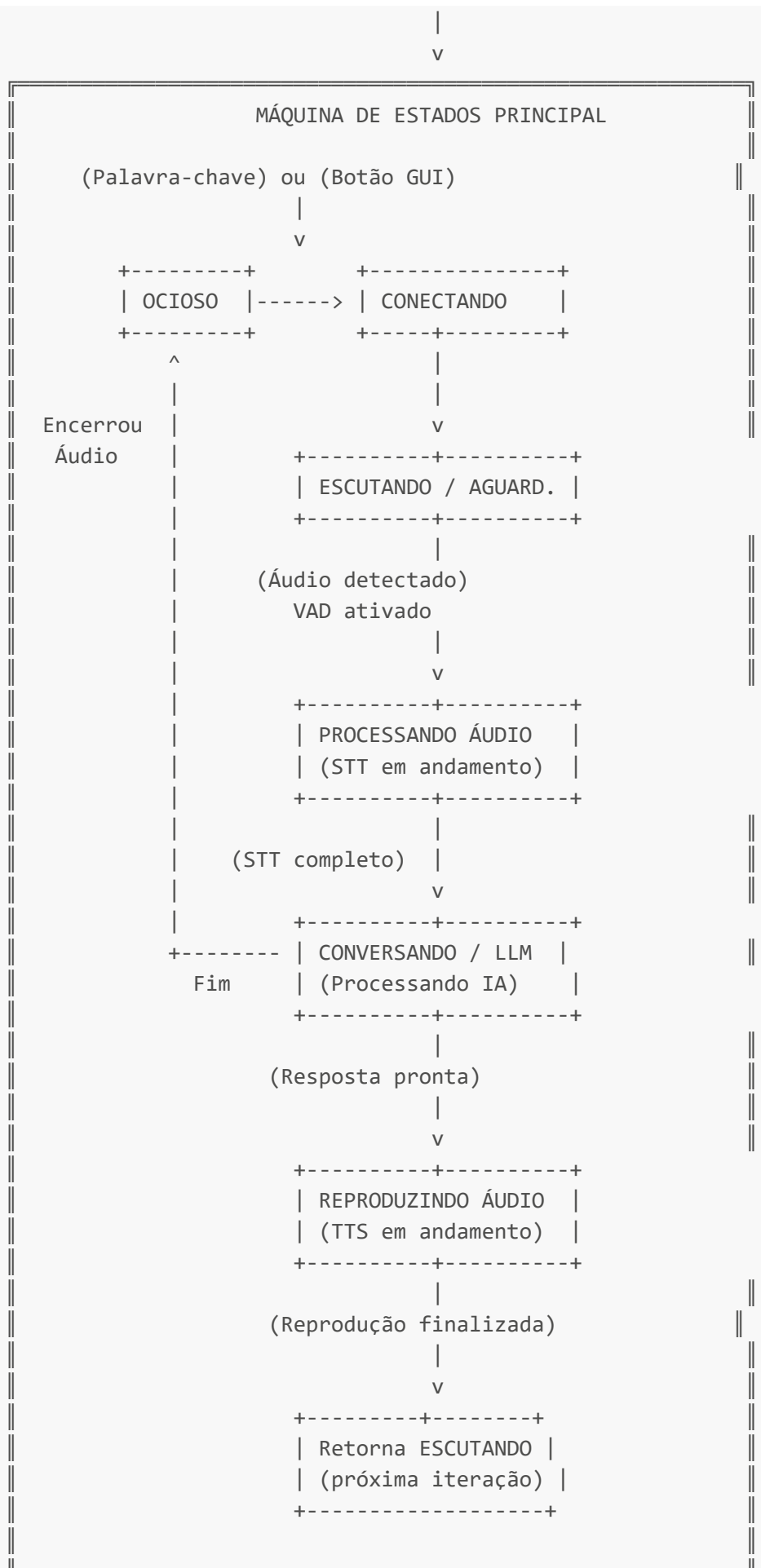
ASSISTENTE-SHI/
├── main.py                # Ponto de entrada
├── src/
│   ├── application.py    # Lógica central
│   ├── audio_codecs/     # Codificação de áudio
│   ├── audio_processing/ # Processamento de áudio
│   ├── core/             # Componentes principais
│   ├── display/          # Camada abstrata de interface
│   ├── iot/              # Gerenciamento IoT
│   ├── mcp/              # Sistema de ferramentas MCP
│   ├── protocols/        # Protocolos de comunicação
│   ├── utils/            # Funções utilitárias
│   └── views/            # Componentes de UI
├── config/               # Arquivos de configuração
├── models/               # Modelos de IA
├── logs/                 # Arquivos de log
└── requirements.txt      # Dependências Python

```

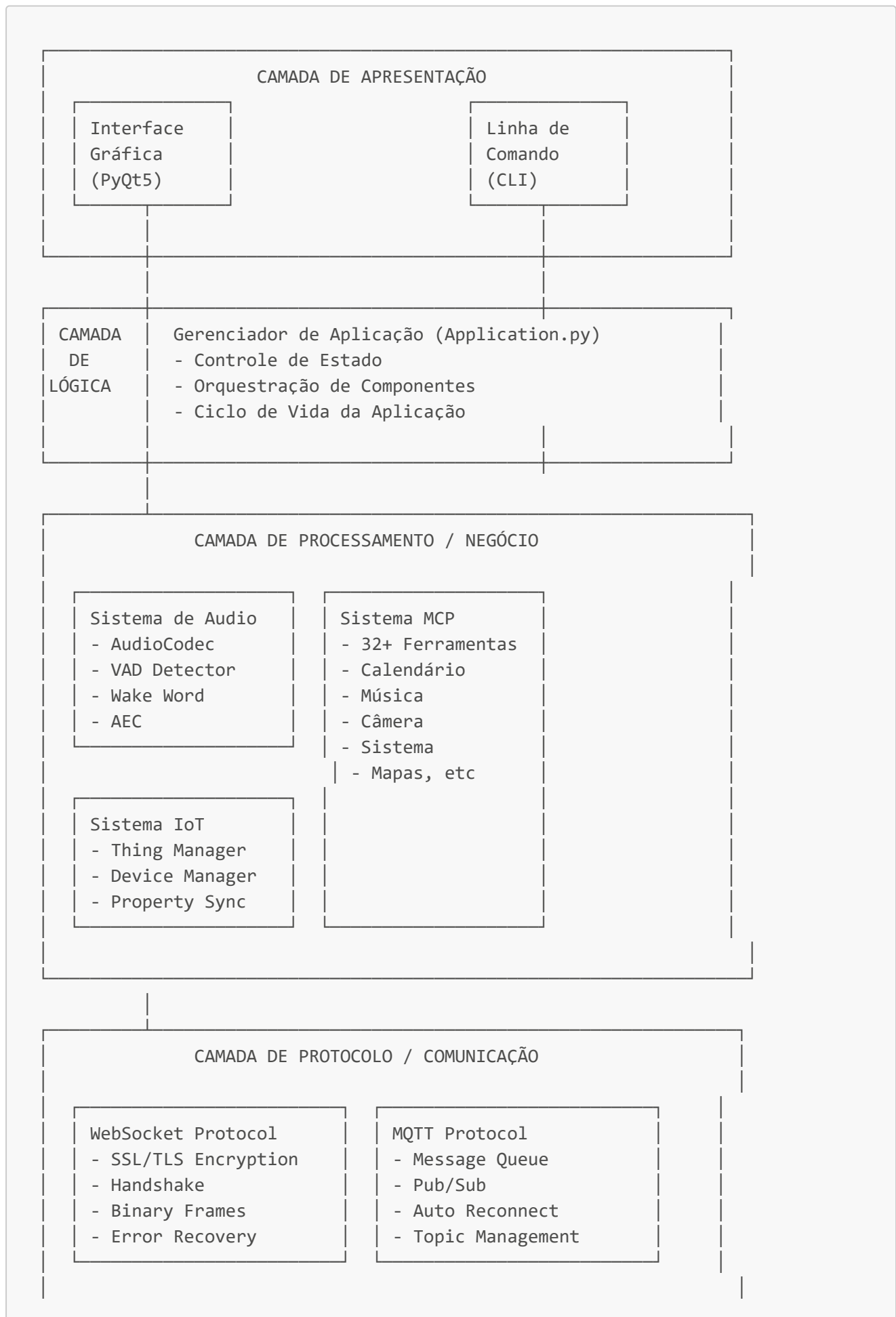
🔄 Fluxos e Diagramas do Projeto

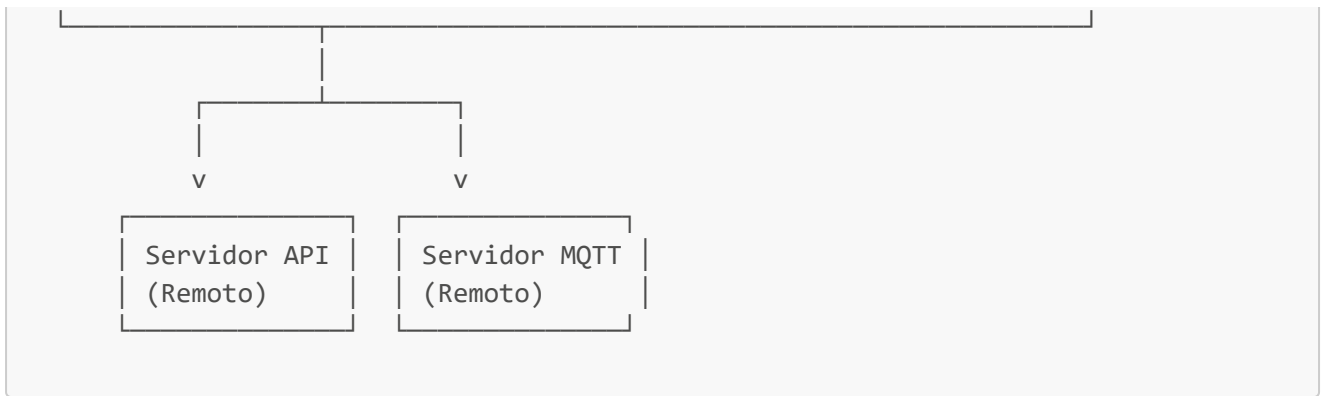
1 Fluxo de Estados da Aplicação



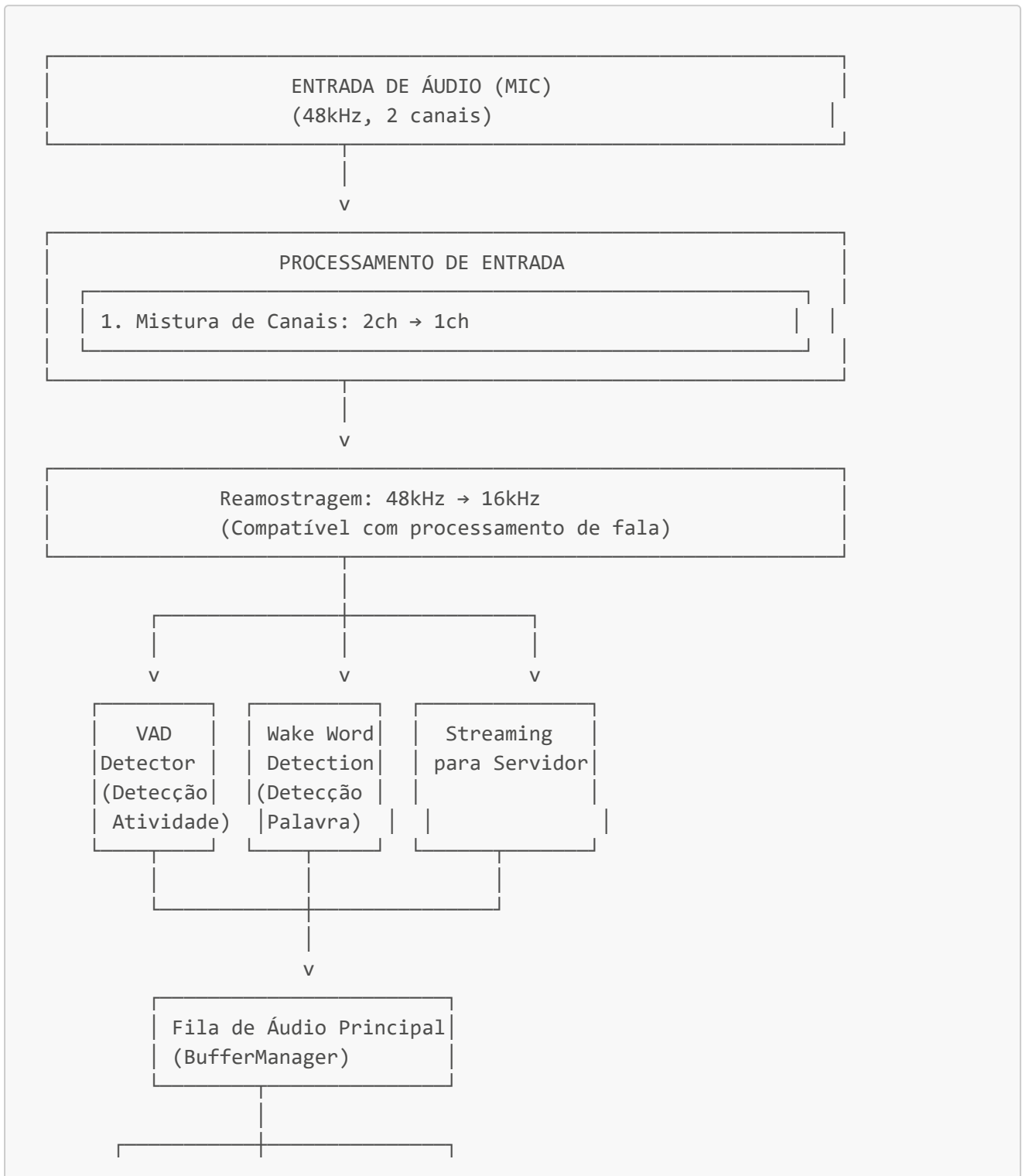


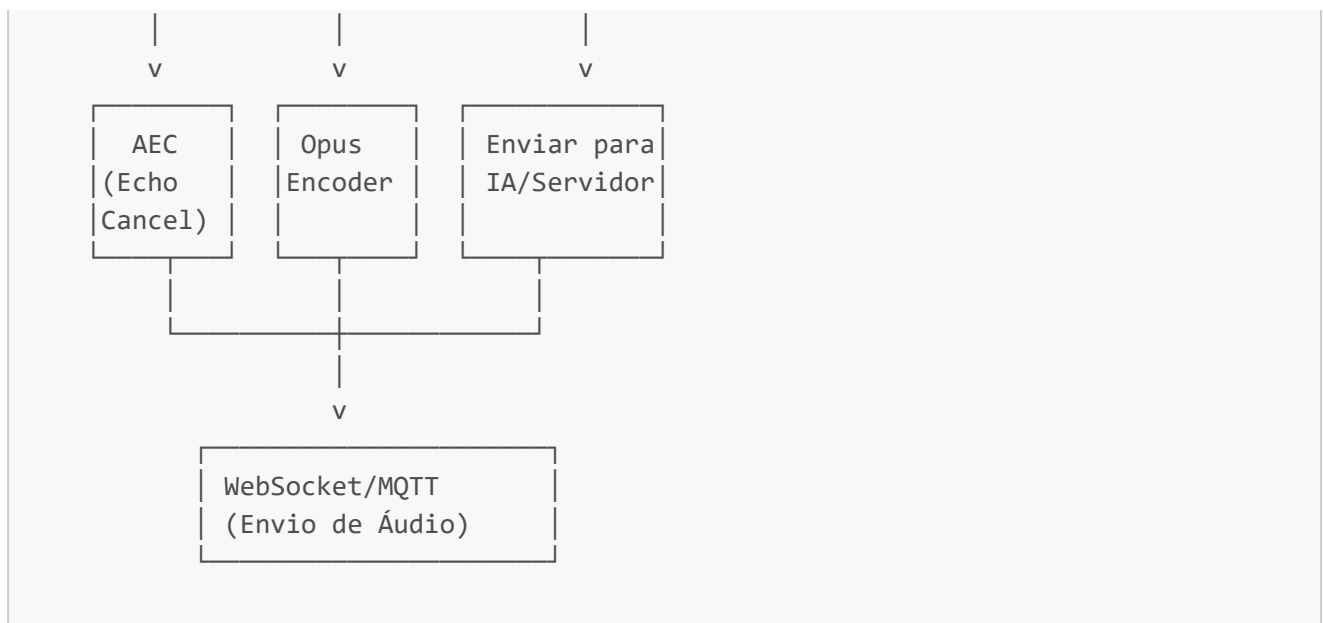
2 Arquitetura em Camadas do Sistema



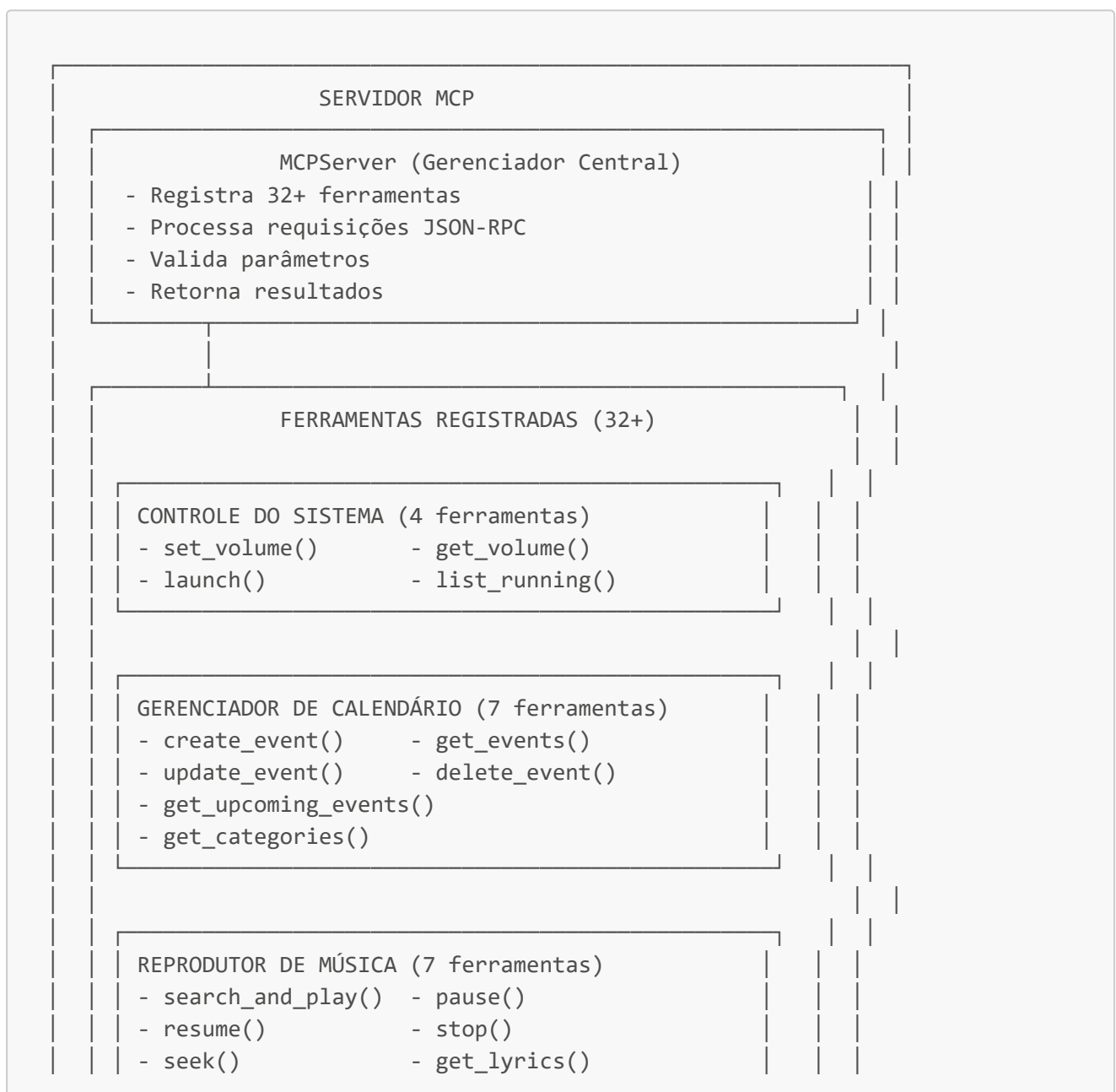


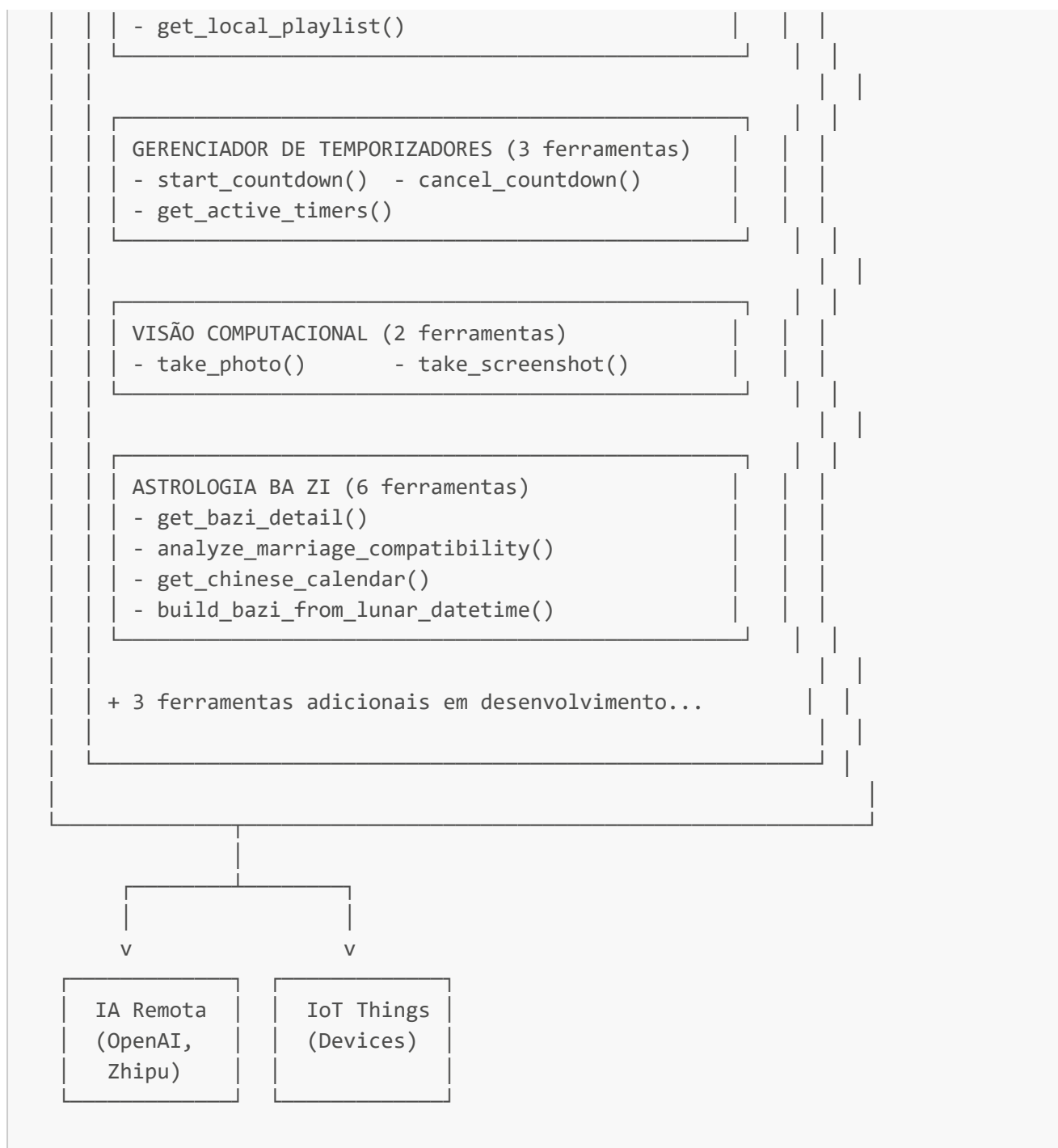
3 Fluxo de Processamento de Áudio



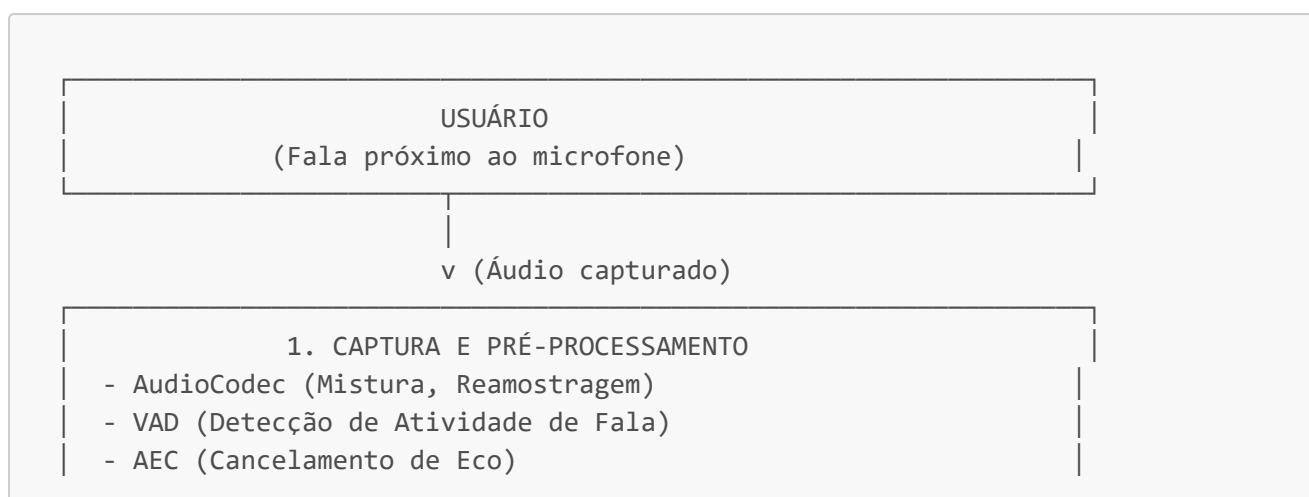


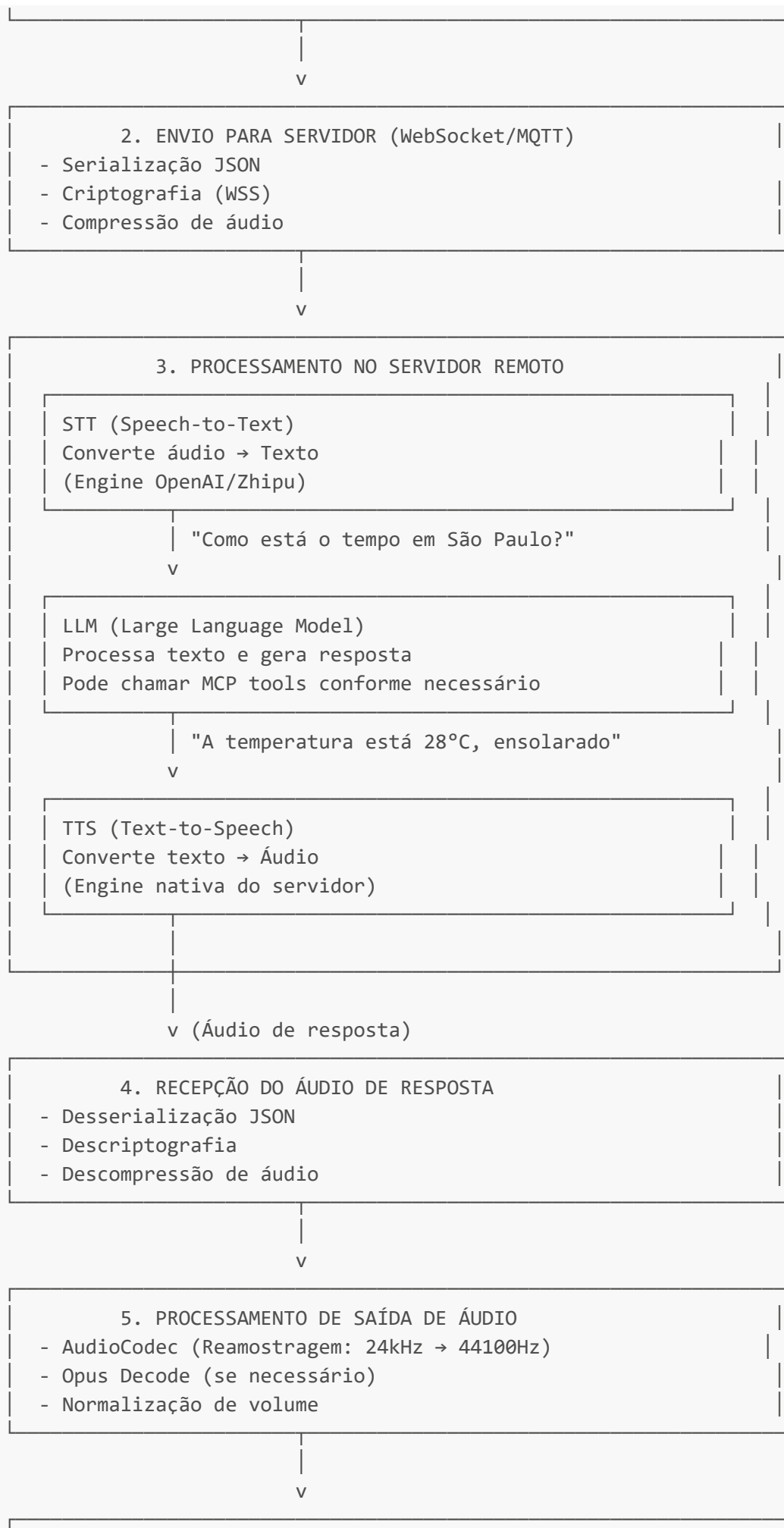
4 Arquitetura do Sistema MCP (Model Context Protocol)





5 Fluxo Completo de Interação Usuário → IA → Resposta





6. REPRODUÇÃO DE ÁUDIO (Speaker)

- Streaming para saída de áudio
- Sincronização com interface
- Feedback visual (animação)

v

USUÁRIO OUVI A RESPOSTA
(Sistema retorna ao estado ESCUTANDO)

6 Fluxo de Inicialização do Sistema

```
python main.py --mode gui --protocol websocket
```

v

parse_args()
(Processar argumentos CLI)

v

setup_logging()
(Iniciar sistema de logs)

v

Detectar Wayland/X11
(Configurar variáveis QT)

v

Criar qasync event loop
(Loop asyncio para Qt5)

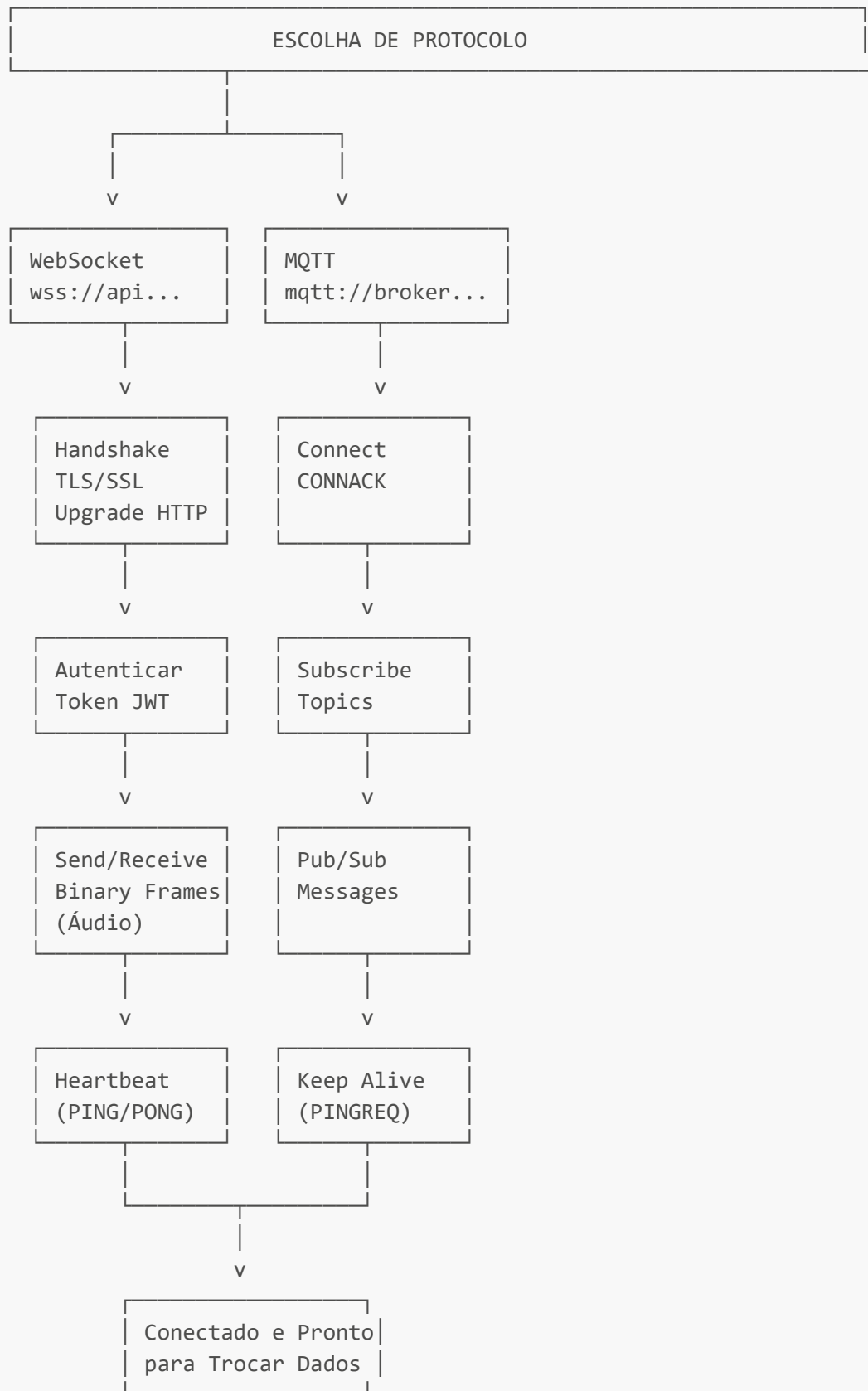
v

handle_activation()
(Processar ativação do dispositivo)

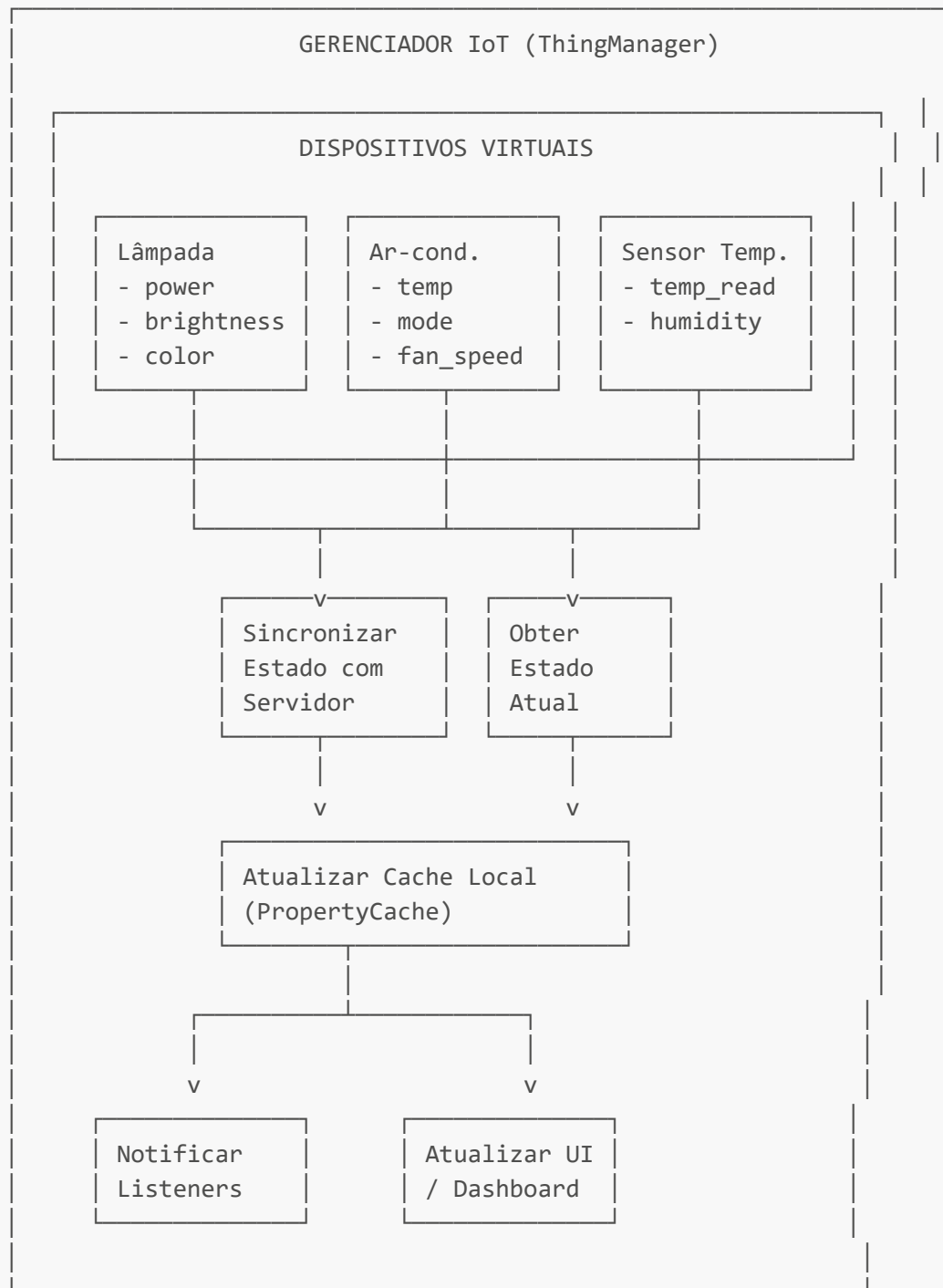
1. SystemInitializer init
2. Gerar Fingerprint
3. Validar/Criar efuse.json
4. Buscar configuração OTA
5. Verificar WebSocket/MQTT



7 Fluxo de Protocolo de Comunicação



8 Fluxo de Sincronização de Estado IoT



Status Atual

☒ Funcional

- ☒ Interface GUI com WebSocket
- ☒ 32 ferramentas MCP registradas
- ☒ Processamento de áudio Opus
- ☒ Interação com IA em tempo real
- ☒ Suporte a câmera (captura de fotos)
- ☒ Gerenciamento de calendário

- ☒ Sistema de lembretes

⚠ Conhecido

- ⚠ Modelo de Wake Word ausente (Sherpa-ONNX)
- ⚠ API de Visão Remota retornando 404
- ⚠ Timeout de conexão WebSocket após ~1 minuto em teste

🔧 Desenvolvimento

Executar em Diferentes Modos

```
# Modo GUI com WebSocket (padrão)
python main.py --mode gui --protocol websocket

# Modo CLI com MQTT
python main.py --mode cli --protocol mqtt

# Pular processo de ativação (debug)
python main.py --skip-activation
```

Adicionando Novas Ferramentas MCP

Estenda a classe em `src/mcp/tools/` e registre em `mcp_server.py`.

Adicionando Novos Dispositivos IoT

Implemente `Thing` base class em `src/iot/things/`.

📊 Estatísticas do Projeto

- **Ferramentas MCP:** 32+ funções integradas
- **Compatibilidade:** 3 sistemas operacionais
- **Protocolos:** 2 opções (WebSocket, MQTT)
- **Modos:** 2 interfaces (GUI, CLI)

🤝 Contribuições

Contribuições são bem-vindas! Por favor:

1. Faça um fork do repositório
2. Crie uma branch para sua feature (`git checkout -b feature/AmazingFeature`)
3. Commit suas mudanças (`git commit -m 'Add some AmazingFeature'`)
4. Push para a branch (`git push origin feature/AmazingFeature`)
5. Abra um Pull Request

📄 Licença

[MIT License](#) - Veja o arquivo LICENSE para detalhes



Agradecimentos

Agradecimentos especiais ao projeto original [py-xiaozhi](#) e todos os contribuidores.



Contato e Suporte

Para dúvidas, sugestões ou relatórios de bugs, abra uma issue no repositório GitHub.

Última atualização: 13 de janeiro de 2026

Status: ☒ Operacional em modo GUI + WebSocket

